

Keywords: post-training quantization • multi-precision LLM
• residual quantization • efficient inference

Recurrent Residual Quantization: A Progressive Multi-Precision Representation for LLMs

One Checkpoint, Four Bit-Widths: A Calibration-Free Framework that Builds 2-, 4-, 6-, and 8-Bit Models from a Single Residual Stack

By Yu Luo, Bo Dong, Wenhua Cheng, Haihao Shen

arXiv:2608.04048 • NeurIPS 2026

Deploying LLMs across heterogeneous hardware—from cloud GPUs to edge devices—today requires a separate quantized checkpoint for every target bit-width. Recurrent Residual Quantization (RRQ) eliminates this overhead by representing weights as a base plus a sequence of 2-bit residual corrections: reading only the base gives 2-bit inference; reading base plus k residuals gives $(2 + 2k)$ -bit inference. The full 2/4/6/8-bit package for Qwen3-8B is constructed in 1,293 seconds, $3.3\times$ faster than MatGPTQ, with no calibration data required.

AT A GLANCE

Problem: Each target bit-width requires a separate quantization run and checkpoint, multiplying storage and maintenance costs linearly with the number of deployment targets.

Why it matters: A single multi-precision checkpoint enables hardware-adaptive serving: the same model file serves data-center GPUs at 8-bit and edge CPUs at 2-bit.

Method: Progressive 2-bit residuals applied to the quantization error of the previous step; no calibration data; no joint multi-bit optimization.

Result: $3.3\times$ faster construction than MatGPTQ; competitive perplexity at all four precisions; single-file deployment.

Evidence: Benchmarked on Qwen3-8B and LLaMA-3-8B; accepted to NeurIPS 2026.

The Big Picture

Large language models are increasingly deployed across a spectrum of hardware: high-memory data-center GPUs running 8- or 16-bit inference, on-device accelerators requiring 4-bit, and low-power edge CPUs that may only support 2-bit integer arithmetic. Current practice requires a separate post-training quantization (PTQ) run—often

including an expensive calibration step—for each target precision. A team deploying Llama-3 at 2-, 4-, and 8-bit must run three separate GPTQ pipelines and store three separate checkpoints.

Recurrent Residual Quantization (RRQ) collapses this into one. A single RRQ construction run produces a structured checkpoint from which any of the four target precisions can be read at inference time by selecting how many correction layers to load. There is no trade-off in quality: each precision performs comparably to a dedicated GPTQ checkpoint optimized for that bit-width.

Why Existing Approaches Fall Short

Calibration-based PTQ (GPTQ, AWQ, MatG-PTQ) achieves near-FP16 quality at 4-bit and above, but requires a calibration dataset (typically 128 sequences of 2048 tokens) and second-order Hessian computations that dominate construction time. A new bit-width requires a new run; the runs cannot be composed.

Residual Vector Quantization (RVQ) decomposes weights into a codebook plus residual codebooks, enabling multiple precisions. However, RVQ optimizes all codebooks jointly, which is expensive, and the discrete codebook structure is poorly suited to the per-layer, per-group statistics of LLM weights.

Non-uniform quantization (NF4, SpQR) adapts bit allocation to weight distributions but is fixed to a single precision per run.

RRQ’s key insight is that greedy round-to-nearest residuals—not joint optimization—are sufficient to close the quality gap between adjacent bit-widths, because each residual corrects for the error of the previous quantization step independently.

Core Mechanism

RRQ builds a weight approximation $\hat{w}^{(k)}$ by iteratively quantizing the error of the previous approximation:

$$\hat{w}^{(0)} = Q_2(w), \quad r^{(j)} = w - \hat{w}^{(j-1)}, \quad \hat{w}^{(j)} = \hat{w}^{(j-1)} + Q_2(r^{(j)}),$$

where $Q_2(\cdot)$ denotes round-to-nearest 2-bit scalar quantization with per-group scales computed from the range of the argument. After k iterations, the approximation has effective precision $(2 + 2k)$ bits. Concretely:

- $\hat{w}^{(0)}$: effective 2-bit (base only)
- $\hat{w}^{(1)}$: effective 4-bit (base + 1 residual)
- $\hat{w}^{(2)}$: effective 6-bit (base + 2 residuals)
- $\hat{w}^{(3)}$: effective 8-bit (base + 3 residuals)

The checkpoint stores $\hat{w}^{(0)}$ and $r^{(1)}, r^{(2)}, r^{(3)}$ as separate tensors; inference at precision $(2 + 2k)$ loads and sums $\hat{w}^{(0)}$ through $Q_2(r^{(k)})$.

Technical Formulation

Let $W \in \mathbb{R}^{m \times n}$ be a weight matrix partitioned into groups of size g . For each group $\mathcal{G}_i$, the 2-bit quantizer is:

$$Q_2(x; \mathcal{G}_i) = \text{round}\left(\frac{x - \min(\mathcal{G}_i)}{\max(\mathcal{G}_i) - \min(\mathcal{G}_i)} \cdot 3\right) \cdot \frac{\max(\mathcal{G}_i) - \min(\mathcal{G}_i)}{3} + \min(\mathcal{G}_i)$$

The reconstruction error after k residual layers satisfies:

$$\|w - \hat{w}^{(k)}\|_2^2 = \|r^{(k)} - Q_2(r^{(k)})\|_2^2 \leq \frac{(\Delta^{(k)})^2}{4},$$

where $\Delta^{(k)} = \max(r^{(k)}) - \min(r^{(k)})$ is the range of the k -th residual, which shrinks geometrically with k as the residuals concentrate near zero.

Learning or Inference Procedure

Construction. For each linear layer, the RRQ construction procedure: (1) computes the 2-bit base $\hat{w}^{(0)}$ via RTN with per-group scales; (2) computes residuals $r^{(j)}$ and their 2-bit quantizations sequentially; (3) stores the base and all residuals. No calibration data is touched; construction is embarrassingly parallel across layers.

Inference. A deployment configuration specifies the target precision $b \in \{2, 4, 6, 8\}$. The runtime loads the base and $(b - 2)/2$ residual tensors, sums them, and uses the result as the weight matrix. Memory footprint is proportional to b ; switching precision at inference time requires only loading more or fewer residual tensors, not re-quantizing.

Hardware kernels. Because both the base and each residual are 2-bit integer tensors with per-group scales, existing 2-bit matmul kernels (e.g. INT2 CUDA kernels) apply directly. The multi-residual accumulation adds one extra addition per residual per layer.

What the Guarantee Says

The greedy residual construction gives a monotone quality improvement: adding each residual layer can only reduce or maintain the reconstruction error, never increase it. Formally, $\|w - \hat{w}^{(k)}\|_2 \leq \|w - \hat{w}^{(k-1)}\|_2$ for all k . This ensures that the 4-bit model is always at least as good as the 2-bit model, and the 6-bit model at least as good as the 4-bit model, from the same checkpoint.

Experimental Findings

- **Construction time (Qwen3-8B):** 1,293 seconds for the full 2/4/6/8-bit package, 3.3× faster than MatGPTQ (4,267 seconds).
- **Perplexity (WikiText-2):** RRQ-4bit matches GPTQ-4bit within 0.1 perplexity points on Qwen3-8B; RRQ-8bit is within 0.05 of FP16.

- **Zero-shot accuracy (MMLU, HellaSwag):** <0.5 point degradation at 4-bit vs. FP16 baseline on both Qwen3-8B and LLaMA-3-8B.
- **Storage:** The complete 4-precision package is 1.15× the size of a single 8-bit GPTQ checkpoint.

Ablations and Interpretation

Group size. Larger groups (128 vs. 64) reduce storage but increase 2-bit error; the residual correction partially compensates, narrowing the quality gap between group sizes. At 4-bit and above, group size has negligible effect.

RTN vs. GPTQ base. Starting from a GPTQ-optimized 2-bit base (rather than RTN) yields 0.3–0.5 perplexity improvement at 2-bit but does not meaningfully affect 4-bit or higher precision, confirming that the residuals correct for RTN’s error.

Number of residuals. Quality saturates after 3 residuals (8-bit effective), matching the known empirical ceiling of scalar PTQ beyond 8-bit for LLMs.

Reference

Yu Luo, Bo Dong, Wenhua Cheng, Haihao Shen. **Recurrent Residual Quantization: A Progressive Multi-Precision Representation for LLMs.** arXiv:2608.04048, NeurIPS 2026. <https://arxiv.org/abs/2608.04048>